DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 2: DATA MODELS

Question Bank with Comprehensive Answers — E-R Diagrams, Concepts & Theory

SYLLABUS OVERVIEW — Unit 2: Data Models

Subject: CT 652 | Full Marks: 80 | Pass Marks: 32 | Duration: 3 hrs exam

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages and Relational Model	7 hrs	12
4	Database Constraints and Normalization	6 hrs	12
5	Query Processing and Optimization	4 hrs	8
6	File Structure and Hashing	4 hrs	8
7	Transactions Processing and Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 2 Topics (7 Hours | 12 Marks)

- 2.1 Logical, Physical and Conceptual Data Models
- 2.2 E-R Model
- 2.3 Entities and Entity Sets
- 2.4 Relationship and Relationship Sets
- 2.5 Strong and Weak Entity Sets
- 2.6 Attributes and Keys
- 2.7 E-R Diagram
- 2.8 Alternate Data Models (hierarchical, network, graph)

ER DIAGRAM NOTATION REFERENCE

All ER diagrams in this chapter use standard Chen/Crow's Foot notation. The following table summarizes the symbols used in the textual ER diagrams throughout this document.

Symbol / Notation	Meaning	Example
ENTITY_NAME (rectangle)	Entity set	STUDENT, DOCTOR
(attribute)	Simple attribute	(name), (age)
((derived))	Derived attribute	((age)) derived from DOB
(multi: attribute{})	Multi-valued attribute	(phones{})
[WEAK_ENTITY] (double rect)	Weak entity set	[ROOM], [DEPENDENT]
--1-- or --N--	Relationship cardinality	STUDENT --N-- ENROLS --1-- COURSE
==== (double line)	Total participation	STUDENT ==== ENROLS
---- (single line)	Partial participation	PROFESSOR ---- TEACHES
<RELATIONSHIP>	Regular relationship	<ENROLS>, <WORKS_IN>
<<IDENTIFYING_REL>>	Identifying relationship for weak entity	<<ROOM_OF>>
{KEY_attr}	Primary key attribute (underlined)	{empid}, {sid}
ISA triangle	Generalization / Specialization	PERSON ISA {EMPLOYEE, STUDENT}
AGGR[...]	Aggregation	AGGR[EMPLOYEE <MANAGES> PROJECT]

NOTE: In exam answers, ER diagrams are described textually with entities, attributes, relationships, cardinalities, and participation constraints clearly listed. Always explicitly state all assumptions about mapping constraints.

CHAPTER 2 — Conceptual Questions & Answers

Q: What are data models? Explain various types of data models.
Asked: 2073 Magh (Back Exam) | Marks: [1+3]

Answer

Definition of Data Model

A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and data constraints. It provides a way to describe the design of a database at the logical level, abstracting the underlying physical storage.

Data models serve as the foundation for any database system. They define how data is structured, accessed, and manipulated.

Types of Data Models

1. Conceptual (High-Level / Entity-Based) Data Models

Provide concepts close to the way users perceive data. They use entities, attributes, and relationships to describe the real world.

- Entity-Relationship (ER) Model: Uses entities, attributes, and relationships; represented as ER diagrams. Most widely used for database design.
- Enhanced ER (EER) Model: Extends ER with generalization, specialization, aggregation, and categories.
- Object-Oriented Model: Combines data and behavior into objects with inheritance.

2. Representational (Implementation / Record-Based) Data Models

Provide concepts that may be understood by end users but are not too far from how data is organized in the computer. Used in most commercial DBMSs.

- Relational Model (RM): Data organized in two-dimensional tables (relations). Uses SQL. Most widely used in practice. Example: MySQL, Oracle, PostgreSQL.
- Network Model: Data organized as records and sets (owner-member relationships). Uses pointers. Can represent M:N relationships directly. Less flexible than relational.
- Hierarchical Model: Data organized in a tree structure (parent-child). Each record has exactly one parent. Simple but rigid; poor support for M:N relationships. Example: IBM IMS.

3. Physical (Low-Level) Data Models

Describe how data is actually stored on the computer storage medium. Concerned with file structures, indexing methods, access paths, and record formats. Used only by system programmers and DBAs.

- Examples: heap files, B+ tree files, hash files, clustered files

Comparison: Hierarchical vs Network vs Relational Model

Aspect	Hierarchical	Network	Relational
Structure	Tree (parent-child)	Graph (owner-member sets)	Tables (rows and columns)
M:N relationships	Difficult (needs duplication)	Supported via sets	Naturally supported via foreign keys
Navigation	Parent-to-child traversal	Pointer-based navigation	SQL (declarative)
Data independence	Low	Low	High
Flexibility	Rigid, limited	More flexible than hierarchical	Very flexible
Example	IBM IMS	IDS, IDMS	MySQL, Oracle, PostgreSQL
Query language	DL/I	None standard	SQL (ISO standard)

Graph-Based / Object Data Model

Modern data models also include: Object-Relational Model (SQL:1999 — adds OOP features to relational), Graph Databases (Neo4j — nodes and edges for highly connected data), Document Model (MongoDB — JSON/BSON documents), Key-Value Model (Redis — simple key-value pairs).

Q: Explain the E-R Model. Define: entity, entity set, attribute, key, relationship, relationship set.

Asked: Foundation for all ER diagram questions | Marks: [Conceptual basis]

Answer

Entity-Relationship (ER) Model

The Entity-Relationship (ER) model, proposed by Peter Chen in 1976, is a high-level conceptual data model used for database design. It represents the real-world structure of a database using three main concepts: entities, attributes, and relationships. ER diagrams (ERDs) are the graphical representation of the ER model.

Entity and Entity Set

An entity is a 'thing' or 'object' in the real world that is distinguishable from all other objects. It can be a physical object (a person, a book, a car) or a conceptual object (a course, a company, a job).

An entity set is a set of entities of the same type that share the same properties (attributes). For example, the set of all students in a university is an entity set STUDENT.

- Strong entity: Has a primary key attribute that uniquely identifies it (e.g., STUDENT with sid)
- Weak entity: Does not have a primary key of its own; depends on an identifying/owner entity for its existence and identification (e.g., DEPENDENT depends on EMPLOYEE)

Attributes

An attribute is a property or characteristic that describes an entity or relationship. Types of attributes:

- Simple (atomic) attribute: Cannot be subdivided further. Example: age, salary
- Composite attribute: Can be subdivided into smaller sub-attributes. Example: name (first_name, middle_name, last_name); address (street, city, zip)

- Single-valued attribute: Has a single value for a given entity. Example: age, SSN
- Multi-valued attribute: Can have multiple values for a given entity. Example: phone_numbers, email_addresses. Shown with double ellipse in ER diagram.
- Derived attribute: Value can be computed from other attributes. Example: age derived from date_of_birth. Shown with dashed ellipse.
- Stored attribute: The attribute from which a derived attribute is computed. Example: date_of_birth
- Null attribute: Takes a null value when no applicable value or value is unknown
- Key attribute: Value is unique for each entity in the entity set; used to identify entities. Shown with underline.

Keys

A key is one or more attributes whose combined values uniquely identify each entity in an entity set.

- Super key: Any set of attributes that uniquely identifies an entity. May contain extra attributes.
- Candidate key: A minimal super key — no proper subset is itself a super key. A relation may have multiple candidate keys.
- Primary key: The candidate key chosen by the database designer to uniquely identify entities. Underlined in ER diagrams.
- Partial key (Discriminator): For weak entities, the attribute(s) that partially identify a weak entity within the entity set of its owner. Shown with dashed underline.
- Foreign key: An attribute in one relation that references the primary key of another relation, establishing a link between two tables.

Relationship and Relationship Set

A relationship is an association among two or more entities. For example: student Ram enrolled in course DBMS is a relationship.

A relationship set is a set of relationships of the same type. For example, ENROLS is a relationship set connecting entity sets STUDENT and COURSE.

- Binary relationship: Involves two entity sets (most common). Example: EMPLOYEE works_in DEPARTMENT
- Unary (recursive) relationship: An entity set is related to itself. Example: EMPLOYEE manages EMPLOYEE
- Ternary relationship: Involves three entity sets. Example: SUPPLIER supplies PART to PROJECT

Cardinality Ratios (Mapping Constraints)

- One-to-One (1:1): An entity in A is associated with at most one entity in B, and vice versa. Example: EMPLOYEE manages DEPARTMENT (each employee manages at most one dept, each dept has at most one manager)
- One-to-Many (1:N): An entity in A can be associated with many entities in B, but each entity in B is associated with at most one entity in A. Example: DEPARTMENT has EMPLOYEES
- Many-to-One (N:1): Reverse of 1:N
- Many-to-Many (M:N): An entity in A can be associated with many entities in B and vice versa. Example: STUDENT enrolls in COURSE

Participation Constraints

- Total participation (mandatory): Every entity in the entity set must participate in at least one relationship instance. Shown with double line (====). Example: Every EMPLOYEE must work in a DEPARTMENT

- Partial participation (optional): Some entities may not participate in any relationship instance. Shown with single line (----). Example: Some EMPLOYEES may manage a DEPARTMENT (but not all)

Generalization, Specialization, and Aggregation

Specialization

Specialization is the process of defining a set of subclasses of an entity set (the superclass). The subclasses have additional attributes or participate in additional relationships. It is a top-down approach — from general to specific.

Example: PERSON is the superclass; EMPLOYEE, STUDENT, ALUMNI are subclasses. Each inherits all attributes of PERSON and has its own additional attributes.

Generalization

Generalization is the reverse of specialization — combining several entity sets with common features into a single higher-level entity set. It is a bottom-up approach — from specific to general.

Example: FACULTY and STAFF are generalized into EMPLOYEE.

Aggregation

Aggregation treats a relationship set (along with its participating entity sets) as a higher-level entity set. It is used when a relationship needs to participate in another relationship. Aggregation represents a 'whole-part' type of relationship.

Example: EMPLOYEE works_on PROJECT with a role (this relationship is itself managed by a MANAGER). The relationship WORKS_ON is aggregated to form a higher-level entity that participates in MANAGES.

Q: Explain the different types of attributes. What is a discriminator key?
Asked: 2080 Ashwin (Back Exam); 2074 Bhadra [4] | Marks: [2+2]

Answer

Types of Attributes

An attribute is a property that describes an entity or a relationship. The following are the main types:

Type	Description	Example	ER Notation
Simple (Atomic)	Indivisible; cannot be broken down further	age, salary, empid	Single ellipse
Composite	Can be divided into sub-parts with independent meaning	address (street, city, zip); name (first, last)	Ellipse with sub-ellipses
Single-Valued	Holds exactly one value per entity	SSN, date_of_birth	Single ellipse
Multi-Valued	Can hold multiple values per entity	phone_numbers, email_addresses	Double ellipse
Derived	Value computed from another (stored) attribute	age (from DOB); years_of_service (from hire_date)	Dashed ellipse

Stored	Base attribute from which derived attribute is calculated	date_of_birth (for age)	Single ellipse
Null	Attribute value is unknown or not applicable	middle_name (if none)	Empty value
Key	Uniquely identifies each entity in the entity set	sid for STUDENT, empid for EMPLOYEE	Underlined in ellipse
Partial Key	Partially identifies weak entity within its owner	room_no within a HOTEL	Dashed underline

Discriminator Key (Partial Key)

A discriminator key (also called a partial key) is the set of attributes that can uniquely identify weak entities related to the same owner entity. It does NOT uniquely identify the weak entity by itself — it only discriminates among weak entities belonging to the same owner.

A weak entity is an entity that cannot be uniquely identified by its own attributes alone. It depends on an identifying entity (owner) for its identification and for its existence. The relationship between a weak entity and its owner is called the identifying relationship.

Example of Discriminator Key:

- Entity: DEPENDENT (weak entity) — depends on EMPLOYEE (strong entity/owner)
- DEPENDENT attributes: dep_name, date_of_birth, relationship, gender
- dep_name is the discriminator/partial key — it uniquely identifies a dependent WITHIN the same employee's family
- Two employees could each have a dependent named 'Anita', but within one employee's family, dep_name is unique
- The full identifier = {empid (from EMPLOYEE) + dep_name (discriminator)}

In an ER diagram, the discriminator/partial key is shown with a dashed underline (unlike the solid underline for primary keys), and the weak entity is shown with a double rectangle. The identifying relationship is shown with a double diamond.

Q: Define total participation with an example. Explain weak entity set with an example. What is an identifying relationship?

Asked: 2078 Baishakh (Back Exam); 2078 Poush [8+4] | Marks: [2+2]

Answer

Strong Entity Set

A strong entity set is an entity set that has a primary key — a set of attributes that uniquely identifies each entity in the set without depending on another entity. Every entity in a strong entity set can be uniquely identified using only its own attributes.

- Examples: STUDENT (primary key: sid), EMPLOYEE (primary key: empid), COURSE (primary key: cid)
- Notation: represented by a single rectangle in ER diagrams

Weak Entity Set

A weak entity set is an entity set that does NOT have sufficient attributes to form a primary key on its own. It depends on an identifying entity (also called owner entity) for:

- Existence — a weak entity cannot exist without its owner entity
- Identification — its identity is derived from the owner entity's primary key combined with its own discriminator/partial key

Characteristics of Weak Entity Sets:

- Has a partial key (discriminator) — dashed underline in ER diagram
- Represented by a double rectangle in ER diagram
- Connected to its owner entity via an identifying relationship (double diamond)
- Uses double lines (total participation) to connect to the identifying relationship
- Full primary key = Owner's primary key + Discriminator of weak entity

Classic Example:

- Weak entity: DEPENDENT — represents family members of an employee
- Owner entity: EMPLOYEE (primary key: empid)
- Identifying relationship: HAS (double diamond)
- Discriminator: dep_name
- Full identifier: (empid, dep_name)
- If EMPLOYEE is deleted, all their DEPENDENTs are automatically deleted

```
EMPLOYEE {empid} ========<<HAS>>=======  [DEPENDENT] {dep_name}
                 (total participation          (double rectangle,
                  from DEPENDENT side)          partial key: dashed underline)
```

Another Example: ROOM in a HOTEL

- Weak entity: ROOM (partial key: room_no)
- Owner: HOTEL (primary key: hotel_id)
- Identifying relationship: CONTAINS
- Full identifier: (hotel_id, room_no) — room 101 in Everest Hotel is different from room 101 in Himalayan Hotel

Total Participation (Mandatory Participation)

Total participation means that every entity in the entity set must participate in at least one relationship instance in the relationship set. No entity can exist without being associated with another entity through the relationship.

- Notation: double line (====) connecting the entity set to the relationship in ER diagrams
- Also called: mandatory participation or existence dependency

Examples of Total Participation:

- Every LOAN must be taken by some CUSTOMER — LOAN has total participation in BORROWS
- Every DEPENDENT must belong to an EMPLOYEE — DEPENDENT has total participation in HAS
- Every ORDER must be placed by some CUSTOMER — ORDER has total participation in PLACES

Examples of Partial Participation:

- Not every EMPLOYEE needs to manage a DEPARTMENT — EMPLOYEE has partial participation in MANAGES
- Not every CUSTOMER necessarily has a LOAN — CUSTOMER has partial participation in BORROWS
- Not every PROFESSOR is supervising a PhD student at a given time

Identifying Relationship

An identifying relationship is the relationship set that connects a weak entity set to its owner/identifying entity set. It is represented by a double diamond in ER diagrams. The weak entity always has total participation (double line) in the identifying relationship.

Q: Distinguish between the degree and cardinality of a relationship in an ER diagram. Explain generalization in ER diagram with an example.

Asked: 2077 Chaitra (Regular Exam); 2071 Bhadra [8+4] | Marks: [2+2]

Answer

Degree of a Relationship

The degree of a relationship set is the number of entity sets that participate in it.

- Unary (degree 1): A relationship between instances of the SAME entity set. Also called recursive relationship. Example: EMPLOYEE manages EMPLOYEE (an employee manages other employees). Useful for representing hierarchical structures.
- Binary (degree 2): A relationship between instances of TWO entity sets. Most common type in ER models. Example: STUDENT enrolls_in COURSE.
- Ternary (degree 3): A relationship among instances of THREE entity sets. Example: SUPPLIER supplies PART to PROJECT. Cannot always be replaced by a binary relationship.
- Higher-degree: Rare in practice, but theoretically possible for 4 or more entity sets.

Cardinality of a Relationship (Mapping Cardinality)

The cardinality ratio (mapping cardinality) specifies the maximum number of relationship instances that an entity can participate in. It describes how many entities on one side of a relationship can be associated with entities on the other side.

Cardinality	Meaning	Example
1:1 (One-to-One)	Each entity in A is associated with at most one entity in B, and vice versa	EMPLOYEE manages at most one DEPARTMENT; DEPARTMENT is managed by at most one EMPLOYEE
1:N (One-to-Many)	One entity in A can be associated with many entities in B, but each entity in B with at most one in A	One DEPARTMENT has many EMPLOYEES; each EMPLOYEE belongs to one DEPARTMENT
M:N (Many-to-Many)	An entity in A can be associated with many in B and vice versa	A STUDENT can enroll in many COURSES; a COURSE can have many STUDENTS
M:1 (Many-to-One)	Many entities in A are associated with one entity in B	Many EMPLOYEES work for one DEPARTMENT (same as 1:N from dept side)

Key Difference: Degree vs Cardinality

Aspect	Degree	Cardinality
Definition	Number of entity sets involved in a relationship	Maximum number of instances each entity can participate in
Possible Values	1 (unary), 2 (binary), 3 (ternary), etc.	1:1, 1:N, M:N
Example Question	Is MANAGES a binary or unary relationship?	Is MANAGES 1:1, 1:N, or M:N?
Focus	Structure of relationship	Quantitative constraint on associations

Generalization in ER Diagram

Generalization is a bottom-up design approach in which two or more lower-level entity sets (subclasses) are combined to form a single higher-level entity set (superclass) based on their common attributes and relationships. It abstracts the common features of subclasses.

Properties of Generalization:

- Inheritance: Subclasses inherit all attributes of the superclass
- The higher-level entity set is a generalization of the lower-level entity sets
- Represented by an ISA (is-a) triangle in ER diagrams, with an arrow pointing from subclass to superclass
- Disjoint constraint: An entity can belong to only one subclass (e.g., a person is either a student OR an employee, not both)
- Overlapping constraint: An entity can belong to multiple subclasses (e.g., a person can be both a student AND an employee)
- Completeness constraint: Total (every superclass entity must belong to a subclass) vs Partial (some may not)

Example:

Suppose a university has two employee types — FACULTY (with attributes: rank, publications) and STAFF (with attributes: position, shift). Both share common attributes (empid, name, salary, address). These are generalized into a superclass EMPLOYEE:

```
FACULTY {rank, publications}    STAFF {position, shift}
        \                           /
         \      ISA (generalization)  /
          \                         /
           EMPLOYEE {empid, name, salary, address}
```

The FACULTY and STAFF entity sets share attributes like empid, name, salary which are moved up to the superclass EMPLOYEE. Only FACULTY has 'rank' and 'publications'; only STAFF has 'position' and 'shift'.

Q: What is the importance of aggregation in ER design? Explain with an example.
Asked: 2076 Baisakh (Back Exam) | Marks: [2+2]

Answer

Aggregation in ER Design

Aggregation is an abstraction in the ER model that allows a relationship (along with its participating entities) to be treated as a higher-level entity. It models the concept of a 'relationship between relationships' — where a relationship set needs to participate in another relationship set.

Why Aggregation is Needed:

The basic ER model does not allow relationships to participate in other relationships. When the semantics of the real world require that a relationship itself plays the role of an entity in another relationship, we use aggregation.

Classic Example:

Consider the following scenario: EMPLOYEE works_on PROJECT using a specific MACHINE. If we have a relationship WORKS_ON between EMPLOYEE and PROJECT, and we also need to record which MACHINE is used in that work, we cannot directly connect MACHINE to the WORKS_ON relationship. We use aggregation:

```
EMPLOYEE ----<WORKS_ON>---- PROJECT
                |
            (aggregated as a higher-level entity)
                |
            <USES>
                |
           MACHINE
```

- The WORKS_ON relationship (between EMPLOYEE and PROJECT) is aggregated
- The resulting aggregate entity participates in the USES relationship with MACHINE
- This allows us to record: 'which employee works on which project using which machine'

Another Example: MANAGES relationship

A MANAGER oversees a work performed by an EMPLOYEE on a PROJECT. The relationship between EMPLOYEE and PROJECT (WORKS_ON) is itself managed by a MANAGER. Aggregation is applied on WORKS_ON to create an aggregate, which then participates in OVERSEES with MANAGER.

Importance of Aggregation:

- Eliminates redundancy: Without aggregation, we would need to duplicate entity sets to model the required semantics
- Models higher-order relationships: Captures 'relationships among relationships' which occur frequently in real-world scenarios
- Improves expressiveness: The ER model becomes more powerful and can represent complex business rules
- Avoids information loss: Without aggregation, we might lose context about which specific relationship instance a second-level relationship refers to

Q: Define unary relationship along with example. How do you convert an ER relationship into relation schema? Explain with examples of different cardinalities.
Asked: 2075 Bhadra (Regular Exam) | Marks: [2+4]

Answer

Unary (Recursive) Relationship

A unary relationship (also called a recursive relationship) is a relationship where an entity set is related to itself. The same entity set participates more than once in the relationship, in different roles.

Examples:

- EMPLOYEE manages EMPLOYEE — some employees manage other employees. Roles: 'manager' and 'subordinate'
- PERSON is_parent_of PERSON — a person can be the parent of other persons
- COURSE is_prerequisite_of COURSE — a course may require another course as a prerequisite

```
EMPLOYEE ----<MANAGES>---- EMPLOYEE
(subordinate role)          (manager role)
Each employee has at most one manager (M:1 from subordinate side)
```

Converting ER to Relational Schema

The following rules convert ER diagram constructs into relational table schemas:

Rule 1: Strong Entity Set → Relation

Each strong entity set becomes a relation. Attributes become columns. Primary key attribute becomes the primary key of the relation.

```
Entity: STUDENT {sid, name, age, address}
Schema: STUDENT(sid, name, age, address)    PK: sid
```

Rule 2: Weak Entity Set → Relation

The weak entity set becomes a relation. Its columns = discriminator attributes + primary key of owner entity (as foreign key). Primary key = owner's PK + discriminator.

```
Weak entity: DEPENDENT {dep_name, dob, relationship}
Owner: EMPLOYEE {empid}
Schema: DEPENDENT(empid, dep_name, dob, relationship)
        PK: (empid, dep_name),  FK: empid → EMPLOYEE(empid)
```

Rule 3: 1:1 Relationship → Merge or Foreign Key

Include the primary key of one entity as a foreign key in the other (preferably in the entity with total participation). Or merge both entities into one relation.

```
EMPLOYEE(1) ----<MANAGES>---- (1)DEPARTMENT
Schema Option A: DEPARTMENT(deptid, dname, manager_empid)
                 FK: manager_empid → EMPLOYEE(empid)
Schema Option B: EMPLOYEE(empid, name, manages_deptid)
```

Rule 4: 1:N Relationship → Foreign Key in N-side

Include the primary key of the '1' side entity as a foreign key in the 'N' side entity's relation.

```
DEPARTMENT(1) ----<HAS>---- (N)EMPLOYEE
Schema: EMPLOYEE(empid, name, salary, dept_id)
        FK: dept_id → DEPARTMENT(deptid)
```

Rule 5: M:N Relationship → New Relation

Create a new relation for the relationship. Its attributes = primary keys of both entity sets + any descriptive attributes of the relationship. Primary key = combination of both entity set keys.

```
STUDENT(M) ----<ENROLS>---- (N)COURSE
  Descriptive attribute: grade
Schema: ENROLS(sid, cid, grade)
        PK: (sid, cid),  FK: sid → STUDENT, cid → COURSE
```

Rule 6: Multi-Valued Attribute → New Relation

Create a new relation with the entity's primary key + the multi-valued attribute. Primary key = entity PK + multi-valued attribute.

```
EMPLOYEE has multi-valued attribute: phone_numbers
Schema: EMP_PHONE(empid, phone_no)
        PK: (empid, phone_no),  FK: empid → EMPLOYEE
```

Q: Explain how network data model is different from relational data model.

Asked: 2072 Magh (Back Exam) | Marks: [4]

Answer

Network Data Model

The network data model (proposed by CODASYL committee, 1969) organizes data as a collection of records and links. Records contain data fields, and links (called sets) represent relationships between records. Unlike hierarchical models, a record can have multiple parent records.

- Uses a graph structure where records are nodes and relationships are edges
- Relationships are physically implemented using pointers
- Supports many-to-many relationships via sets
- Navigation is pointer-based — programmer must navigate through chains of pointers
- Examples: IDMS (Integrated Database Management System), IDS

Comparison: Network vs Relational Data Model

Aspect	Network Model	Relational Model
Data Structure	Records linked by pointers (graph)	Tables (relations) with rows and columns
Relationships	Via owner-member sets (pointers)	Via foreign keys and joins
M:N Relationships	Supported via two 1:N sets	Naturally supported with junction table

Query Language	No standard; navigational DML	SQL (ISO standard, declarative)
Navigation	Programmer navigates pointer chains	DBMS handles navigation internally
Data Independence	Low — schema changes break programs	High — programs work despite schema changes
Ease of Use	Complex for users and programmers	Simple, intuitive (table-based)
Performance	Fast for known access paths	Good with optimizer; flexible ad-hoc queries
Flexibility	Limited — fixed relationships at design time	High — any table can be joined to any other
Standardization	CODASYL standard (not widely adopted)	ISO/ANSI SQL standard
Modern Usage	Legacy systems (IDMS)	Ubiquitous — MySQL, Oracle, PostgreSQL

Key Limitation of Network Model:

In the network model, the application program must know the physical structure of the database (which sets exist, the pointer chains) to retrieve data. If the structure changes, all programs must be rewritten. The relational model overcomes this with data independence — the SQL query remains the same regardless of how data is physically stored.

CHAPTER 2 — ER Diagram Questions

Q: Draw an ER-diagram for Hospital Management System.
Asked: 2082 Kartik (Back Exam) | Marks: [8]

Answer

Entities, Attributes, and Relationships

The following entities, attributes, and relationships are identified from the problem statement:

Entities and Attributes:

- DEPARTMENT: {dept_id (PK), dept_name}
- DOCTOR: {doctor_id (PK), name, specialization, experience} — belongs to one department, treats multiple patients
- PATIENT: {patient_id (PK), name, age, gender, medical_history, patient_type (inpatient/outpatient)}
- ROOM: {room_no (PK), type (ICU/General/Private), cost_per_day} — assigned to admitted patients
- NURSE: {employee_id (PK), name, shift_timings} — assists doctors, cares for patients
- MEDICINE: {medicine_id (PK), name, expiry_date} — stored in pharmacy
- APPOINTMENT: {appt_id (PK), date, time} — patient consults doctor

Relationships and Cardinalities:

- BELONGS_TO: DOCTOR (N) → DEPARTMENT (1) — each doctor belongs to one department; a department has many doctors [N:1, total from DOCTOR]
- TREATS: DOCTOR (M) ↔ PATIENT (N) — a doctor can treat many patients; a patient can be treated by many doctors [M:N]
- ASSIGNED_TO: PATIENT (N) → ROOM (1) — admitted patient is assigned one room; one room assigned to one patient at a time [1:1 at a time, partial from PATIENT]
- ASSISTS: NURSE (M) ↔ DOCTOR (N) — nurses assist doctors [M:N]
- PRESCRIBED: DOCTOR prescribes MEDICINE to PATIENT — ternary relationship with attribute 'quantity', 'date' [M:N:M]
- APPOINTMENT: PATIENT (M) ↔ DOCTOR (N) — with attributes date, time [M:N]

ER DIAGRAM — Textual Representation

```
ENTITIES AND ATTRIBUTES:
──────────────────────────────────────────────────────────────────
DEPARTMENT          : {dept_id(PK), dept_name}
DOCTOR              : {doctor_id(PK), name, specialization, experience}
PATIENT             : {patient_id(PK), name, age, gender, medical_history,
                       patient_type}
[ROOM]              : {room_no(partial PK), type, cost_per_day}   -- weak entity
NURSE               : {employee_id(PK), name, shift_timings}
MEDICINE            : {medicine_id(PK), name, expiry_date}
```

```
RELATIONSHIPS AND CARDINALITIES:
──────────────────────────────────────────────────────────────────────

DOCTOR (N) ==BELONGS_TO== (1) DEPARTMENT
  [Total from DOCTOR; each doctor must belong to one department]

DOCTOR (M) --TREATS-- (N) PATIENT
  [Partial both sides; treated_date as relationship attribute]

PATIENT (1) --ASSIGNED_TO-- (1) ROOM
  [Partial from PATIENT — only admitted patients get rooms]
  [ROOM is weak entity; HOSPITAL or DEPARTMENT is owner]

NURSE (M) --ASSISTS-- (N) DOCTOR
  [Partial both sides]

DOCTOR (M) --PRESCRIBES-- (N) MEDICINE -- (to) PATIENT(P)
  [Ternary relationship; attributes: quantity, prescribed_date]

PATIENT (M) --APPOINTMENT-- (N) DOCTOR
  [Relationship attributes: appt_date, appt_time]
```

Assumptions:

- A doctor belongs to exactly one department (total, N:1)
- A nurse can assist multiple doctors and a doctor can be assisted by multiple nurses (M:N)
- Room assignment is only for inpatient patients (partial participation from PATIENT)
- The prescription relationship is ternary — doctor, medicine, and patient are all involved

---

Q: A relational database is to be designed for a medium-sized Company... Construct an ER diagram assuming the necessary attributes, relationship sets and mapping constraints.

Asked: 2081 Chaitra (Regular); 2079 Chaitra (Regular) | Marks: [8]

Answer

Analysis of the Problem

Key facts: The company delivers products and complete hardware+software installations. Employees are experts, consultants, and support staff — all on long-term basis (no short-term/temporary). The company is organized into departments headed by department managers. Projects are interdisciplinary — teams are formed from multiple departments. Each project has a dedicated Project Manager (an employee) who is exclusively responsible for control.

Entities and Attributes:

- EMPLOYEE: {empid(PK), name, address, type (expert/consultant/support), employment_type (always 'long-term')}
- DEPARTMENT: {dept_id(PK), dept_name, location}
- PROJECT: {project_id(PK), title, start_date, end_date, budget}
- CUSTOMER: {cust_id(PK), name, address, contact}

- PRODUCT: {product_id(PK), description, category} — application program, hardware, software

Relationships and Cardinalities:

- WORKS_IN: EMPLOYEE (N) → DEPARTMENT (1) — each employee works in one department [N:1, total from EMPLOYEE]
- HEADS: EMPLOYEE (1) → DEPARTMENT (1) — one employee heads each department [1:1, partial from EMPLOYEE]
- WORKS_ON: EMPLOYEE (M) ↔ PROJECT (N) — employees from different depts work on projects [M:N]; attribute: hours
- MANAGES: EMPLOYEE (1) → PROJECT (1) — one Project Manager per project, exclusively responsible [1:1]; attribute: responsibility
- ASSIGNED_TO: PROJECT (M) ↔ DEPARTMENT (N) — projects involve multiple departments [M:N]
- DELIVERS: PROJECT (M) ↔ CUSTOMER (N) — project delivers product/service to customer [M:N]

ER DIAGRAM — Textual Representation

```
ENTITIES:
EMPLOYEE {empid(PK), name, address, type}
DEPARTMENT {dept_id(PK), dept_name, location}
PROJECT {project_id(PK), title, start_date, end_date, budget}
CUSTOMER {cust_id(PK), name, address, contact}

RELATIONSHIPS:
EMPLOYEE (N) ==WORKS_IN== (1) DEPARTMENT
  [Total from EMPLOYEE — every employee is in a department]

EMPLOYEE (1) --HEADS-- (1) DEPARTMENT
  [Partial from EMPLOYEE — only managers head departments]
  [1:1 — one manager per department, one department per manager]

EMPLOYEE (M) --WORKS_ON-- (N) PROJECT
  [M:N — team from multiple departments; attribute: hours_worked]
  [Partial both sides]

EMPLOYEE (1) --MANAGES-- (1) PROJECT
  [1:1 — exactly one Project Manager per project]
  [Partial from EMPLOYEE; Total from PROJECT — every project has a PM]
  [Relationship attribute: responsibility (exclusively responsible)]

PROJECT (M) --INVOLVES-- (N) DEPARTMENT
  [M:N — interdisciplinary projects span multiple departments]

PROJECT (M) --DELIVERS_TO-- (N) CUSTOMER
  [Attribute: delivery_type (product/software/hardware installation)]
```

Key Design Decisions:

- The Project Manager (PM) is also an employee — hence MANAGES is a relationship from EMPLOYEE to PROJECT
- MANAGES is 1:1 and total from PROJECT — every project must have exactly one PM

- The PM is exclusively responsible, independent of the company hierarchy — hence a separate MANAGES relationship
- WORKS_ON is M:N — multiple employees from different departments work on one project

---

Q: A Home Decor company sells different furniture and has salespersons... Construct an ER diagram assuming the necessary attributes, relationship sets and mapping constraints.

Asked: 2081 Ashwin (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- SALESPERSON: {sp_id(PK), name, commission, city}
- CUSTOMER: {cust_id(PK), cust_name, city, grade}
- ORDER: {order_id(PK), purchase_amount, order_date}
- FURNITURE: {item_id(PK), brand_name(unique identifier), description, price}
- SUPPLIER: {supplier_id(PK), name, address, phone_number}
- PAYMENT: {payment_id(PK), amount, payment_date, month}

Relationships and Cardinalities:

- SUPERVISES: SALESPERSON (1) ↔ SALESPERSON (N) — recursive; a SP can manage many others [1:N unary]
- SELLS_TO: SALESPERSON (1) ↔ CUSTOMER (N) — each customer managed by one SP; SP sells to many customers [1:N]
- PLACES: CUSTOMER (1) → ORDER (N) — customer places many orders [1:N, total from ORDER]
- CONTAINS: ORDER (M) ↔ FURNITURE (N) — an order lists many furniture items; item in many orders [M:N]; attribute: quantity
- SUPPLIES: SUPPLIER (M) ↔ FURNITURE (N) — one supplier can supply many brands; one brand from one supplier [M:N or 1:N; the problem says 'supplier supplies furniture from many brands' so M:N]
- PAYS: COMPANY (or PURCHASE entity) → SUPPLIER; multiple payments per month [attribute: month, payment_date, amount]

ER DIAGRAM — Textual Representation

```
SALESPERSON {sp_id(PK), name, commission, city}
CUSTOMER {cust_id(PK), cust_name, city, grade}
ORDER {order_id(PK), purchase_amount, order_date}
FURNITURE {item_id(PK), brand_name, description, price}
SUPPLIER {supplier_id(PK), name, address, phone}

SALESPERSON (1) --SUPERVISES-- (N) SALESPERSON  [unary, 1:N]

SALESPERSON (1) --MANAGES-- (N) CUSTOMER
  [Each customer managed by one SP; partial from SALESPERSON]
```

```
CUSTOMER (1) ==PLACES== (N) ORDER
  [Total from ORDER; every order placed by one customer]

ORDER (M) ==CONTAINS== (N) FURNITURE
  [Total from ORDER; attribute: quantity]

SUPPLIER (M) --SUPPLIES-- (N) FURNITURE
  [Partial both sides; brand_name identifies furniture uniquely per supplier]

SUPPLIER --RECEIVES_PAYMENT-- (attribute: amount, month, payment_date)
  [Multiple payments per month; PAYMENT as a weak entity of SUPPLIER
   or as a relationship attribute set]
```

Q: Design a ER diagram for a database that keeps track of three types of persons: employees, alumni, and students. A person can belong to one, two, or all three of these types...

Asked: 2080 Ashwin (Back Exam) | Marks: [2+2+8]

Answer

Analysis

This is a generalization/specialization scenario with OVERLAPPING participation (a person can be employee AND student AND alumni simultaneously). Subtypes have their own specific attributes.

Entities and ISA Hierarchy:

- Superclass: PERSON {SSN(PK), name, sex, address, birth_date}
- Subclass EMPLOYEE: {salary} — further classified as FACULTY, STAFF, STUDENT_ASSISTANT
- - FACULTY: {rank}
- - STAFF: {position}
- - STUDENT_ASSISTANT: further divided into RESEARCH_ASSISTANT {research_project} and TEACHING_ASSISTANT {current_course}
- Subclass ALUMNI: {degree_name, year_granted, major_department}
- Subclass STUDENT: {major_department} — further GRADUATE {degree_program(MS/PhD/MBA)} or UNDERGRADUATE {class(freshman/sophomore)}

ER DIAGRAM — Textual Representation

```
SUPERCLASS: PERSON {SSN(PK), name, sex, address, birth_date}
  ISA (overlapping, partial) — a person can be in multiple subclasses
  |
  |── EMPLOYEE {salary}
  |     ISA (disjoint, total for employees)
  |     |── FACULTY {rank}
  |     |── STAFF {position}
  |     └── STUDENT_ASSISTANT
  |               ISA (disjoint)
  |               |── RESEARCH_ASSISTANT {research_project}
```

```
   |                └── TEACHING_ASSISTANT {current_course}
   |
   |── ALUMNI {degree_name, year_granted, major_department}
   |
   └── STUDENT {major_department}
             ISA (disjoint)
             |── GRADUATE {degree_program}  (M.S., Ph.D., M.B.A.)
             └── UNDERGRADUATE {class}  (freshman, sophomore, etc.)

RELATIONSHIPS:
EMPLOYEE (N) ==WORKS_IN== (1) DEPARTMENT
EMPLOYEE (1) --HEADS-- (1) DEPARTMENT  [partial from EMPLOYEE]
STUDENT (M) --ENROLLED_IN-- (N) COURSE
ALUMNI --GRADUATED_FROM-- DEPARTMENT
```

ISA Constraints:

- PERSON → {EMPLOYEE, ALUMNI, STUDENT}: OVERLAPPING (a person can be in multiple subclasses) and PARTIAL (some persons may not be in any subclass)
- EMPLOYEE → {FACULTY, STAFF, STUDENT_ASSISTANT}: DISJOINT (an employee is exactly one type) and TOTAL (every employee is exactly one of these three)
- STUDENT_ASSISTANT → {RESEARCH_ASSISTANT, TEACHING_ASSISTANT}: DISJOINT and TOTAL
- STUDENT → {GRADUATE, UNDERGRADUATE}: DISJOINT and TOTAL

Q: Draw an ER diagram for the mini-case: Procurement department of an Organization (ABC) keeps track of all the items such as Furniture and equipment...
Asked: 2079 Ashwin (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- ITEM: {item_id(PK), item_name, type (furniture/equipment), purchase_date} — assigned unique ID when purchased
- BUILDING: {building_id(PK), building_name}
- ROOM: [ROOM] {room_no(partial PK), floor, capacity} — weak entity; depends on BUILDING
- DEPARTMENT: {dept_id(PK), dept_name, location}

Relationships:

- LOCATED_IN: ITEM (N) → ROOM (1) — each item assigned to one room [N:1, total from ITEM]
- CONTAINS: BUILDING (1) → ROOM (N) — a building has many rooms [1:N, total from ROOM; ROOM is weak entity of BUILDING]
- ASSIGNED_TO: ROOM (N) → DEPARTMENT (1) — each room assigned to one department [N:1]

ER DIAGRAM — Textual Representation

```
ITEM {item_id(PK), item_name, type, purchase_date}
BUILDING {building_id(PK), building_name}
[ROOM] {room_no(partial PK), floor, capacity}   -- weak entity of BUILDING
DEPARTMENT {dept_id(PK), dept_name}

ITEM (N) ==LOCATED_IN== (1) ROOM
  [Total from ITEM — every item is in exactly one room]

BUILDING (1) ==<<CONTAINS>>== (N) [ROOM]
  [Identifying relationship; ROOM is weak entity of BUILDING]
  [Full PK of ROOM = (building_id, room_no)]

ROOM (N) --ASSIGNED_TO-- (1) DEPARTMENT
  [Each room assigned to one department; partial from ROOM]
```

Q: Explain the distinctions among the terms primary key, candidate key and super key... Design an Entity-Relationship diagram for the ONLINE AUCTION database.
Asked: 2079 Jestha (Back Exam) | Marks: [3+8]

Answer — Key Types

Super Key, Candidate Key, Primary Key:

- Super Key: Any set of one or more attributes that uniquely identifies a tuple in a relation. May have extra (unnecessary) attributes. Example: {sid}, {sid, name}, {sid, name, age} are all super keys of STUDENT if sid is unique.
- Candidate Key: A minimal super key — no proper subset of it is also a super key. A relation may have multiple candidate keys. Example: STUDENT may have both {sid} and {email} as candidate keys if both uniquely identify students.
- Primary Key: One candidate key chosen by the designer to be the principal identifier. Used in foreign key references. Cannot be NULL. Underlined in ER diagrams.

Key Type	Definition	Properties
Super Key	Set of attributes that uniquely identifies a row	May contain redundant attributes; every candidate key is a super key
Candidate Key	Minimal super key	No proper subset is itself a super key; may have multiple per relation
Primary Key	Chosen candidate key	One per relation; used for references; cannot be NULL
Foreign Key	Attribute(s) referencing PK of another relation	Enforces referential integrity; can be NULL (if optional)

ER Diagram for Online Auction System

Entities and Attributes:

- MEMBER: {member_no(PK), email, name, password, home_address, phone} — can be buyer and/or seller

- BUYER: {shipping_address} — subclass of MEMBER
- SELLER: {bank_account_no, routing_no} — subclass of MEMBER
- ITEM: {item_no(PK), title, description, start_bid_price, bid_increment, start_date, end_date, category}
- BID: {bid_id(PK), bid_price, bid_time} — weak entity; made by buyer on item
- CATEGORY: {category_id(PK), category_name, parent_category} — hierarchical classification
- TRANSACTION: {transaction_id(PK), final_price, transaction_date}
- FEEDBACK: {feedback_id(PK), rating(1-10), comment, feedback_date}

Relationships:

- MEMBER → ISA → {BUYER, SELLER}: OVERLAPPING (a member can be both buyer and seller)
- PLACES: BUYER (M) ↔ ITEM (N) via BID — BUYER (1) places (N) BIDs; BID references (N) ITEMS [M:N via BID]
- LISTS: SELLER (1) → ITEM (N) — seller lists items for auction [1:N, total from ITEM]
- BELONGS_TO: ITEM (N) → CATEGORY (1) — each item belongs to one category [N:1]
- WINS: BUYER (1) ← ITEM (1) — item is won by one buyer (highest bidder) [1:1, partial]
- RESULTS_IN: TRANSACTION between BUYER and SELLER for an ITEM [ternary]
- GIVES: MEMBER (M) ↔ MEMBER (N) via FEEDBACK — buyer and seller give each other feedback post-transaction

ER DIAGRAM — Textual Representation

```
MEMBER {member_no(PK), email, name, password, home_address, phone}
  ISA (overlapping)
  |── BUYER {shipping_address}
  └── SELLER {bank_account_no, routing_no}

ITEM {item_no(PK), title, description, start_bid, bid_increment,
      start_date, end_date}
CATEGORY {category_id(PK), category_name, parent_category_id}
[BID] {bid_price, bid_time}  -- weak entity; PK=(member_no, item_no, bid_time)
TRANSACTION {trans_id(PK), final_price, trans_date}
FEEDBACK {feedback_id(PK), rating, comment, feedback_date}

SELLER (1) ==LISTS== (N) ITEM  [total from ITEM]

ITEM (N) --BELONGS_TO-- (1) CATEGORY

BUYER (1) ==PLACES== (N) [BID] on ITEM (1)
  [BID is weak; identifying relationship is PLACED_ON]

ITEM (1) --WON_BY-- (1) BUYER  [partial; item may not be won if no bids]

BUYER + SELLER + ITEM ===RESULTS_IN=== TRANSACTION  [ternary]
  [attributes: final_price, transaction_date]

MEMBER (M) --GIVES_FEEDBACK-- (N) MEMBER
  [recursive M:N; FEEDBACK as associative entity
   attributes: rating(1-10), comment, feedback_date]
```

Q: An University Registrar's Office maintains data about Courses, Course Offerings, Students, and Instructors... Construct an E-R diagram for the Registrar's office.
Asked: 2078 Chaitra (Regular Exam) | Marks: [8]

Answer

Entities and Attributes:

- COURSE: {course_no(PK), title, credits, syllabus, prerequisites{}}
- COURSE_OFFERING: {offering_id(PK), year, semester, section_no, timings, classroom} — weak entity of COURSE
- STUDENT: {student_id(PK), name, program}
- INSTRUCTOR: {instructor_id(PK), name, department, title}

Relationships:

- OFFERS: COURSE (1) → COURSE_OFFERING (N) [1:N, total from OFFERING; identifying relationship]
- TEACHES: INSTRUCTOR (M) ↔ COURSE_OFFERING (N) [M:N — multiple instructors can teach same offering; optional]
- ENROLLED_IN: STUDENT (M) ↔ COURSE_OFFERING (N) [M:N]; attribute: grade
- PREREQUISITE: COURSE (M) ↔ COURSE (N) [recursive M:N — a course can have many prerequisites and be a prerequisite for many]

ER DIAGRAM — Textual Representation

```
COURSE {course_no(PK), title, credits, syllabus}
  Multi-valued: prerequisites{}

[COURSE_OFFERING] {section_no(partial PK), year, semester, timings, classroom}
  -- weak entity of COURSE; PK = (course_no, section_no, year, semester)

STUDENT {student_id(PK), name, program}
INSTRUCTOR {instructor_id(PK), name, department, title}

COURSE (1) ==<<OFFERS>>== (N) [COURSE_OFFERING]
  [identifying relationship; total from OFFERING]

INSTRUCTOR (M) --TEACHES-- (N) COURSE_OFFERING
  [Partial both sides; M:N]

STUDENT (M) ==ENROLLED_IN== (N) COURSE_OFFERING
  [attribute: grade; total from ENROLLED_IN perspective]

COURSE (M) --IS_PREREQUISITE_OF-- (N) COURSE  [recursive M:N]
```

Q: Draw an ER-diagram for: Each employee in an engineering company has at most one recognized skill, but a given skill may be possessed by several employees. An employee can operate a machine-type if they have the associated skill. Each skill is associated with only one machine type. What is a weak entity set and identifying relationship?

Asked: 2078 Poush (Back); 2070 Bhadra (Regular) | Marks: [8+4]

Answer

Entities and Attributes:

- EMPLOYEE: {empid(PK), name, address, salary}
- SKILL: {skill_id(PK), skill_name, description}
- MACHINE_TYPE: {machine_type_id(PK), type_name (lathe/grinder/welder), description}

Relationships:

- HAS_SKILL: EMPLOYEE (N) → SKILL (1) — each employee has at most one skill; each skill can be held by many employees [N:1, partial from EMPLOYEE]
- ASSOCIATED_WITH: SKILL (N:1) → MACHINE_TYPE (1) — each skill is associated with exactly one machine type; one machine type may require multiple skills [N:1, total from SKILL]
- CAN_OPERATE: EMPLOYEE → MACHINE_TYPE — derived from HAS_SKILL + ASSOCIATED_WITH; not stored separately
- MAINTAINS: EMPLOYEE (M) ↔ MACHINE_TYPE (N) — an employee with the skill can maintain those machine types [M:N]; attribute: last_maintenance_date

ER DIAGRAM — Textual Representation

```
EMPLOYEE {empid(PK), name, address, salary}
SKILL {skill_id(PK), skill_name, description}
MACHINE_TYPE {machine_type_id(PK), type_name, description}

EMPLOYEE (N) --HAS_SKILL-- (0..1) SKILL
  [Partial from EMPLOYEE — employee may have at most one skill]
  [Partial from SKILL — not all skills held by some employee now]

SKILL (N) ==ASSOCIATED_WITH== (1) MACHINE_TYPE
  [Total from SKILL — each skill linked to exactly one machine type]
  [Partial from MACHINE_TYPE — one machine type may require multiple skills]

EMPLOYEE (M) --MAINTAINS-- (N) MACHINE_TYPE
  [An employee can maintain a machine type only if they have the
   required skill — this is a business rule/constraint]
  [Attribute: last_maintenance_date]
```

Weak Entity Set and Identifying Relationship (Theory Part)

(See C4 above for complete explanation. Summary:)

- Weak entity set: An entity set without a primary key of its own. Depends on identifying (owner) entity for existence and identification.
- Examples: DEPENDENT depends on EMPLOYEE; ROOM depends on HOTEL; COURSE_OFFERING depends on COURSE
- Identifying relationship: The relationship between a weak entity and its owner. Shown with double diamond in ER diagram.
- Discriminator (partial key): Attribute that distinguishes weak entities within the same owner. Shown with dashed underline.
- Full PK of weak entity = Owner's PK + Discriminator

Q: A publishing company produces books on various subjects... Draw ER-diagram for above scenario.

Asked: 2078 Baishakh (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- BOOK: {isbn(PK), title, subject, publication_date} — uniquely identified by Bookid (isbn)
- AUTHOR: {author_id(PK), name, specialization_subject} — specializes in one particular subject
- EDITOR: {editor_id(PK), name, department} — not necessarily a specialist; sole responsibility for one or more publications
- SUPPLIER: {supplier_id(PK), name, address, phone} — supplies items for publication
- ITEM: {item_id(PK), description, unit_of_measure} — items needed for publication (paper, ink, etc.)
- SHOP_OWNER: {shop_id(PK), name, address} — buys books from publisher; one shop can buy many books, one book can only be bought by one shop owner

Relationships:

- WRITTEN_BY: BOOK (N) ↔ AUTHOR (1) — each book written by authors with one specialization [N:1 or M:N depending on interpretation]
- EDITED_BY: EDITOR (1) → BOOK (N) — each editor solely responsible for editing one or more books [1:N]
- REQUIRES: BOOK (M) ↔ ITEM (N) — each book requires items for publication [M:N]
- SUPPLIED_BY: ITEM (N) ↔ SUPPLIER (M) — one supplier can supply many items [M:N]
- PURCHASED_BY: BOOK (N) → SHOP_OWNER (1) — one book bought by one shop owner only [N:1]

ER DIAGRAM — Textual Representation

```
BOOK {isbn(PK), title, subject, publication_date}
AUTHOR {author_id(PK), name, specialization_subject}
EDITOR {editor_id(PK), name, department}
SUPPLIER {supplier_id(PK), name, address, phone}
ITEM {item_id(PK), description, unit_of_measure}
SHOP_OWNER {shop_id(PK), name, address}

AUTHOR (M) --WRITES-- (N) BOOK  [M:N — a book has one or more authors]

EDITOR (1) ==SOLELY_EDITS== (N) BOOK
  [Total from BOOK; each book has exactly one editor responsible]

BOOK (M) --REQUIRES-- (N) ITEM
  [Each book needs several items; items used for many books]

SUPPLIER (M) --SUPPLIES-- (N) ITEM
  [One supplier can supply many items; one item from many suppliers]

BOOK (N) --PURCHASED_BY-- (1) SHOP_OWNER
  [One book can be bought by only one shop owner (problem statement)]
  [One shop owner can buy many books]
```

Q: Draw an ER diagram for an online book store that contains books written by authors and published by the publisher... Also the book is stocked in the warehouse.
Asked: 2077 Chaitra (Regular Exam) | Marks: [8]

Answer

Entities and Attributes:

- BOOK: {isbn(PK), price, title, year}
- AUTHOR: {author_id(PK), name, url, address, email}
- PUBLISHER: {publisher_id(PK), name, address, phone, url}
- CUSTOMER: {cust_id(PK), name, address, email, phone}
- WAREHOUSE: {warehouse_id(PK), code, phone, address}
- SHOPPING_CART: {cart_id(PK), created_date} — weak entity of CUSTOMER

Relationships:

- WRITTEN_BY: BOOK (M) ↔ AUTHOR (N) — a book can have multiple authors; an author can write multiple books [M:N]
- PUBLISHED_BY: BOOK (N) → PUBLISHER (1) — a book published by one publisher [N:1, total from BOOK]
- STOCKED_IN: BOOK (M) ↔ WAREHOUSE (N) — books stocked in warehouses [M:N]; attribute: quantity
- CONTAINS: SHOPPING_CART (M) ↔ BOOK (N) — customer adds books to cart [M:N]; attribute: quantity
- PLACED_BY: SHOPPING_CART (N) → CUSTOMER (1) — each cart belongs to one customer [N:1]
- PURCHASES: CUSTOMER (M) ↔ BOOK (N) — customer purchases books online [M:N]; attribute: purchase_date, amount

ER DIAGRAM — Textual Representation

```
BOOK {isbn(PK), price, title, year}
AUTHOR {author_id(PK), name, url, address, email}
PUBLISHER {publisher_id(PK), name, address, phone, url}
CUSTOMER {cust_id(PK), name, address, email, phone}
WAREHOUSE {warehouse_id(PK), code, phone, address}
[CART] {cart_id(partial PK), created_date}  -- weak entity of CUSTOMER

BOOK (M) ==WRITTEN_BY== (N) AUTHOR  [M:N]

BOOK (N) ==PUBLISHED_BY== (1) PUBLISHER  [N:1, total from BOOK]

BOOK (M) --STOCKED_IN-- (N) WAREHOUSE
  [attribute: quantity_in_stock]

CUSTOMER (1) ==<<OWNS>>== (N) [CART]
  [Identifying relationship; Cart cannot exist without Customer]

[CART] (M) --CONTAINS-- (N) BOOK
  [attribute: quantity, added_date]

CUSTOMER (M) --PURCHASES-- (N) BOOK
  [attributes: purchase_date, amount, transaction_id]
```

Q: A company having a chain of pharmacies wishes you to design a database... Draw an E-R diagram for the given case.
Asked: 2076 Baisakh (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- PATIENT: {ssn(PK), name, address, age} — has one primary physician
- DOCTOR: {ssn(PK), name, specialty, years_of_experience}
- PHARMACEUTICAL_COMPANY: {company_id(PK), name, phone}
- DRUG: {drug_id(PK), trade_name, formula} — trade_name unique per company
- PHARMACY: {pharmacy_id(PK), name, address, phone}
- PRESCRIPTION: {presc_id(PK), date, quantity} — associative entity for doctor prescribing drug to patient
- CONTRACT: {contract_id(PK), start_date, end_date, contract_text, supervisor}

Relationships:

- PRIMARY_PHYSICIAN: PATIENT (N) → DOCTOR (1) — each patient has one primary physician [N:1, total from PATIENT]
- PRESCRIBES: DOCTOR (M) ↔ DRUG (N) to PATIENT (P) — ternary; attribute: date, quantity; last prescription overwrites
- SOLD_BY: DRUG (M) ↔ PHARMACY (N) — a pharmacy sells many drugs at a price; same drug may vary in price by pharmacy [M:N]; attribute: price
- PRODUCES: PHARMACEUTICAL_COMPANY (1) → DRUG (N) — company produces drugs; trade_name unique per company [1:N]; if company deleted, drugs deleted (participation dependency)
- CONTRACT: PHARMACEUTICAL_COMPANY (M) ↔ PHARMACY (N) — long-term contracts [M:N]; attributes: start_date, end_date, text, supervisor

ER DIAGRAM — Textual Representation

```
PATIENT {ssn(PK), name, address, age}
DOCTOR {ssn(PK), name, specialty, years_experience}
PHARM_COMPANY {company_id(PK), name, phone}
DRUG {drug_id(PK), trade_name, formula}
  [trade_name is unique within each company — composite candidate key:
(company_id, trade_name)]
PHARMACY {pharmacy_id(PK), name, address, phone}

PATIENT (N) ==HAS_PRIMARY_PHYSICIAN== (1) DOCTOR
  [Total from PATIENT — every patient has a primary physician]

PHARM_COMPANY (1) ==PRODUCES== (N) DRUG
  [Total from DRUG; if company deleted → drugs deleted (total participation)]

DOCTOR (M) --PRESCRIBES-- (N) DRUG --to-- (P) PATIENT  [TERNARY]
  [Represented as PRESCRIPTION {presc_id, date, quantity}
   FK: doctor_ssn, drug_id, patient_ssn; only last prescription stored]

DRUG (M) --SOLD_AT-- (N) PHARMACY
  [attribute: price (varies by pharmacy)]
```

```
PHARM_COMPANY (M) --CONTRACT-- (N) PHARMACY
  [attributes: start_date, end_date, contract_text, supervisor]
  [Supervisor can change over lifetime of contract]
```

Q: Identify relevant attributes and construct an ER diagram with proper mapping constraints for a university which has many departments, each department has multiple instructors, one among them is the head. An instructor belongs to only one department, each department offers multiple courses, each of which is taught by a single instructor. A student may enroll for many courses offered by different departments.

Asked: 2075 Bhadra (Regular Exam) | Marks: [6]

Answer

Entities and Attributes:

- DEPARTMENT: {dept_id(PK), dept_name, location}
- INSTRUCTOR: {instructor_id(PK), name, title, email, specialization}
- COURSE: {course_id(PK), course_name, credits, description}
- STUDENT: {student_id(PK), name, program, year_of_study, email}

Relationships:

- BELONGS_TO: INSTRUCTOR (N) → DEPARTMENT (1) — each instructor in one dept [N:1, total from INSTRUCTOR]
- HEADS: INSTRUCTOR (1) → DEPARTMENT (1) — one instructor is head of dept [1:1, partial from INSTRUCTOR, total from DEPARTMENT]
- OFFERS: DEPARTMENT (1) → COURSE (N) — each course offered by one dept [1:N]
- TEACHES: INSTRUCTOR (1) → COURSE (1) — each course taught by a single instructor [1:1, total from COURSE]
- ENROLLS_IN: STUDENT (M) ↔ COURSE (N) — student enrolls in courses from different depts [M:N]; attribute: grade

ER DIAGRAM — Textual Representation

```
DEPARTMENT {dept_id(PK), dept_name, location}
INSTRUCTOR {instructor_id(PK), name, title, email}
COURSE {course_id(PK), course_name, credits}
STUDENT {student_id(PK), name, program, year_of_study}

INSTRUCTOR (N) ==BELONGS_TO== (1) DEPARTMENT
  [Total from INSTRUCTOR — every instructor belongs to exactly one dept]

INSTRUCTOR (1) --HEADS-- (1) DEPARTMENT
  [Partial from INSTRUCTOR — only head instructors participate]
  [Total from DEPARTMENT — every dept must have a head]

DEPARTMENT (1) ==OFFERS== (N) COURSE
  [Total from COURSE — every course offered by some department]
```

```
INSTRUCTOR (1) ==TEACHES== (1) COURSE
  [Each course taught by exactly one instructor]
  [Total from COURSE — every course must have an instructor]
  [Partial from INSTRUCTOR — not every instructor teaches every course]

STUDENT (M) ==ENROLLS_IN== (N) COURSE
  [M:N; attribute: grade, enrollment_date]
```

Q: Construct an ER-Diagram for the following NFL database. The NFL has many teams, each team has a name, city, coach, a captain and a set of players. Each player belongs to only one team and each player has a name, position, skill level, and injury records...

Asked: 2075 Baisakh (Back Exam) | Marks: [8+4]

Answer

Entities and Attributes:

- TEAM: {team_id(PK), name, city, coach} — has a captain (one player)
- PLAYER: {player_id(PK), name, position, skill_level} — belongs to one team
- INJURY_RECORD: {injury_id(PK), date, description, recovery_time} — weak entity of PLAYER
- MATCH: {match_id(PK), match_date, score_host, score_guest}

Relationships:

- BELONGS_TO: PLAYER (N) → TEAM (1) — each player on one team [N:1, total from PLAYER]
- IS_CAPTAIN: PLAYER (1) → TEAM (1) — one captain per team; captain is a player [1:1, partial from PLAYER]
- SUFFERS: PLAYER (1) → INJURY_RECORD (N) — player can have multiple injuries [1:N]; INJURY_RECORD is weak entity
- PLAYS: TEAM (M) ↔ MATCH (N) — two teams per match (host and guest) [M:N]; roles: host_team, guest_team; attribute: score

ER DIAGRAM — Textual Representation

```
TEAM {team_id(PK), name, city, coach}
PLAYER {player_id(PK), name, position, skill_level}
[INJURY_RECORD] {injury_id(partial PK), date, description, recovery_time}
  -- weak entity of PLAYER; PK = (player_id, injury_id)
MATCH {match_id(PK), match_date}

PLAYER (N) ==BELONGS_TO== (1) TEAM
  [Total from PLAYER — every player belongs to exactly one team]

PLAYER (1) --IS_CAPTAIN_OF-- (1) TEAM
  [Partial from PLAYER — only captain participates]
  [Total from TEAM — every team must have a captain]
```

```
PLAYER (1) ==<<HAS>>== (N) [INJURY_RECORD]
  [Identifying relationship; INJURY_RECORD is weak entity of PLAYER]

TEAM (M) --PLAYS_IN-- (N) MATCH
  [Each match involves exactly 2 teams: host_team and guest_team]
  [Roles: host, guest; attributes: score_host, score_guest]
  [Total from MATCH — every match has exactly 2 teams]
```

Strong vs Weak Entity Sets (Theory Part)

- Strong entity set: Has sufficient attributes to form a primary key. Example: PLAYER {player_id(PK), name, ...} — player_id uniquely identifies each player globally.
- Weak entity set: Does NOT have a primary key of its own; depends on a strong (owner) entity. Example: INJURY_RECORD depends on PLAYER — injury_id alone does not uniquely identify a record across all players, only within one player's records. Full PK = (player_id, injury_id).

Q: Draw the Entity-Relationship Diagram (ERD) with appropriate mapping cardinalities for a Production company (machining, fabrication, assembly departments, employees with skills, machines, raw materials from vendors).

Asked: 2074 Bhadra (Regular Exam) | Marks: [8]

Answer

Entities and Attributes:

- DEPARTMENT: {dept_id(PK), dept_name (machining/fabrication/assembly), location}
- EMPLOYEE: {empid(PK), name, designation (mechanic/welder etc.), payment_basis (daily/regular), overtime_rate}
- SKILL: {skill_id(PK), skill_name}
- MACHINE_TYPE: {machine_id(PK), type_name, department_id}
- PRODUCT: {product_id(PK), product_type, MRP, specifications}
- PART: {part_id(PK), description, source_dept}
- RAW_MATERIAL: {material_id(PK), name, unit}
- VENDOR: {vendor_id(PK), name, address, contact}

Relationships:

- ASSIGNED_TO: EMPLOYEE (N) → DEPARTMENT (1) [N:1, total from EMPLOYEE]
- MANAGED_BY: DEPARTMENT (N) → MANAGER/EMPLOYEE (1) [N:1; each dept has one manager]
- HAS_SKILL: EMPLOYEE (M) ↔ SKILL (N) [M:N; employee may have multiple skills]
- OPERATES: EMPLOYEE (M) ↔ MACHINE_TYPE (N) [M:N; if employee has required skill]
- MAINTAINS: EMPLOYEE (M) ↔ MACHINE_TYPE (N) [M:N; based on required skill]
- ASSEMBLED_FROM: PRODUCT (M) ↔ PART (N) [M:N; many parts form a product]
- USED_IN: PART (M) ↔ PRODUCT (N) [M:N]
- SUPPLIED_BY: RAW_MATERIAL (M) ↔ VENDOR (N) [M:N]
- USES_MATERIAL: PART/PRODUCT manufacturing uses RAW_MATERIAL [M:N]

ER DIAGRAM — Textual Representation

```
DEPARTMENT {dept_id(PK), dept_name, location}
EMPLOYEE {empid(PK), name, designation, payment_basis, overtime_rate}
SKILL {skill_id(PK), skill_name}
MACHINE_TYPE {machine_id(PK), type_name}
PRODUCT {product_id(PK), product_type, MRP}
PART {part_id(PK), description}
RAW_MATERIAL {material_id(PK), name, unit}
VENDOR {vendor_id(PK), name, address, contact}

EMPLOYEE (N) ==ASSIGNED_TO== (1) DEPARTMENT
EMPLOYEE (1) --MANAGES-- (1) DEPARTMENT  [partial from EMPLOYEE]
EMPLOYEE (M) --HAS_SKILL-- (N) SKILL
EMPLOYEE (M) --OPERATES-- (N) MACHINE_TYPE  [requires skill]
MACHINE_TYPE (N) --BELONGS_TO-- (1) DEPARTMENT
PRODUCT (M) --ASSEMBLED_FROM-- (N) PART
PART (M) --MADE_FROM-- (N) RAW_MATERIAL
RAW_MATERIAL (M) --SUPPLIED_BY-- (N) VENDOR
  [attribute: price, delivery_time]
```

Q: Differentiate total and partial participation with suitable example and draw an ER diagram for the airport database: airplanes, models, technicians (expertise in plane models), traffic controllers (annual medical examination).

Asked: 2073 Bhadra (Regular Exam) | Marks: [4+8]

Answer — Total vs Partial Participation

(See C4 above for complete theory. Summary:)

- Total participation: Every entity must participate. Notation: double line (====). Example: Every AIRPLANE must be of some MODEL — AIRPLANE has total participation in IS_OF relationship with AIRPLANE_MODEL.
- Partial participation: Entities may or may not participate. Notation: single line (----). Example: Not every TECHNICIAN necessarily has expertise in every airplane model — TECHNICIAN has partial participation in EXPERT_IN.

ER Diagram — Airport Database

Entities and Attributes:

- AIRPLANE: {reg_no(PK), status}
- AIRPLANE_MODEL: {model_no(PK), capacity, weight}
- TECHNICIAN: {empid(PK), name, SSN, address, phone, salary}
- TRAFFIC_CONTROLLER: {empid(PK), name, certification_date, work_shift} — subclass of EMPLOYEE; must have annual medical exam
- MEDICAL_EXAM: {exam_id(PK), exam_date, result, doctor_name} — most recent exam per traffic controller

Relationships:

- IS_OF_MODEL: AIRPLANE (N) → AIRPLANE_MODEL (1) — each airplane is a specific model [N:1, total from AIRPLANE]

- EXPERT_IN: TECHNICIAN (M) ↔ AIRPLANE_MODEL (N) — a technician can have expertise in multiple models; a model needs multiple technicians [M:N, partial both]
- HAS_EXAM: TRAFFIC_CONTROLLER (1) → MEDICAL_EXAM (1) — most recent medical exam [1:1 for current; can be 1:N for history]

ER DIAGRAM — Textual Representation

```
AIRPLANE {reg_no(PK), status}
AIRPLANE_MODEL {model_no(PK), capacity, weight}
TECHNICIAN {empid(PK), name, SSN, address, phone, salary}
TRAFFIC_CONTROLLER {empid(PK), name, certification_date, work_shift}
MEDICAL_EXAM {exam_id(PK), exam_date, result, doctor_name}

AIRPLANE (N) ==IS_OF_MODEL== (1) AIRPLANE_MODEL
  [Total from AIRPLANE — every airplane has a model]

TECHNICIAN (M) --EXPERT_IN-- (N) AIRPLANE_MODEL
  [Partial both sides; M:N]
  [One technician can overlap expertise with other technicians on same model]

TRAFFIC_CONTROLLER (1) ==HAS_CURRENT_EXAM== (1) MEDICAL_EXAM
  [Total from TRAFFIC_CONTROLLER — must have annual exam record]
  [The most recent exam is stored; previous exams may be archived]

TECHNICIAN and TRAFFIC_CONTROLLER can be modeled as subtypes of EMPLOYEE
  EMPLOYEE {empid(PK), name, SSN, address, phone, salary}
    ISA (disjoint)
    |-- TECHNICIAN {}
    └-- TRAFFIC_CONTROLLER {certification_date, work_shift}
```

Q: Design an E-R diagram for an Airlines system. The database must keep track of customers, their reservations, flights and their status, seat assignments on individual flights, and the schedule and routing of future flights.

Asked: 2073 Magh (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- CUSTOMER: {cust_id(PK), name, email, phone, address, passport_no}
- FLIGHT: {flight_no(PK), status (scheduled/delayed/cancelled/arrived), departure_time, arrival_time, duration}
- AIRCRAFT: {aircraft_id(PK), model, capacity, airline}
- AIRPORT: {airport_code(PK), name, city, country}
- RESERVATION: {reservation_id(PK), booking_date, seat_no, class, ticket_price}
- ROUTE: {route_id(PK), distance, frequency}

Relationships:

- MAKES: CUSTOMER (1) → RESERVATION (N) — customer makes reservations [1:N, total from RESERVATION]

- FOR_FLIGHT: RESERVATION (N) → FLIGHT (1) — reservation for specific flight [N:1, total from RESERVATION]
- OPERATES: AIRCRAFT (1) → FLIGHT (N) — aircraft operates flights [1:N, total from FLIGHT]
- DEPARTS_FROM: FLIGHT (N) → AIRPORT (1) — flight departs from airport [N:1, total from FLIGHT]
- ARRIVES_AT: FLIGHT (N) → AIRPORT (1) — flight arrives at airport [N:1, total from FLIGHT]
- FOLLOWS: FLIGHT (N) → ROUTE (1) — flight follows a route [N:1]
- CONNECTS: ROUTE connects AIRPORT (source) to AIRPORT (destination) [binary relationship with roles]

ER DIAGRAM — Textual Representation

```
CUSTOMER {cust_id(PK), name, email, phone, passport_no}
FLIGHT {flight_no(PK), status, departure_time, arrival_time}
AIRCRAFT {aircraft_id(PK), model, capacity}
AIRPORT {airport_code(PK), name, city, country}
RESERVATION {reservation_id(PK), booking_date, seat_no, class, price}
ROUTE {route_id(PK), distance, frequency}

CUSTOMER (1) ==MAKES== (N) RESERVATION  [total from RESERVATION]
RESERVATION (N) ==FOR_FLIGHT== (1) FLIGHT  [total from RESERVATION]
AIRCRAFT (1) ==OPERATES== (N) FLIGHT  [total from FLIGHT]
FLIGHT (N) ==DEPARTS_FROM== (1) AIRPORT  [role: departure_airport]
FLIGHT (N) ==ARRIVES_AT== (1) AIRPORT  [role: arrival_airport]
FLIGHT (N) --FOLLOWS-- (1) ROUTE
ROUTE: SOURCE_AIRPORT(1) --CONNECTS-- DESTINATION_AIRPORT(1)
  [ROUTE has FK: source_airport_code, destination_airport_code]
```

Q: Draw an ER-diagram for: Patients are treated in a single ward by doctors assigned to them. Healthcare assistants also attend patients; a number are associated with each ward. Each patient takes a variety of drugs a certain number of times per day. Treatments are required for which patients. Staff payment includes overtime.

Asked: 2072 Ashwin (Regular Exam) | Marks: [8+4]

Answer

Entities and Attributes:

- PATIENT: {patient_id(PK), name, age, admission_date, diagnosis}
- WARD: {ward_id(PK), ward_name, capacity, floor}
- DOCTOR: {doc_id(PK), name, specialization, salary}
- HEALTHCARE_ASSISTANT: {ha_id(PK), name, shift, overtime_rate, hourly_rate}
- DRUG: {drug_id(PK), name, description, dosage_form}
- TREATMENT: {treatment_id(PK), treatment_name, description, cost}

Relationships:

- ADMITTED_IN: PATIENT (N) → WARD (1) — patient in one ward [N:1, total from PATIENT]
- ASSIGNED_TO: DOCTOR (M) ↔ WARD (N) — doctors assigned to wards [M:N]; a doctor can be in multiple wards, a ward has multiple doctors

- ASSOCIATED_WITH: HEALTHCARE_ASSISTANT (M) ↔ WARD (N) — HAs associated with wards [M:N]
- TREATED_BY: PATIENT (M) ↔ DOCTOR (N) — patients treated by doctors [M:N]
- PRESCRIBED_DRUG: PATIENT takes DRUG — M:N; attributes: times_per_day, duration, start_date
- REQUIRES_TREATMENT: PATIENT (M) ↔ TREATMENT (N) — treatments required for patients [M:N]; attribute: date, result

ER DIAGRAM — Textual Representation

```
PATIENT {patient_id(PK), name, age, admission_date, diagnosis}
WARD {ward_id(PK), ward_name, capacity, floor}
DOCTOR {doc_id(PK), name, specialization, salary}
HEALTHCARE_ASSISTANT {ha_id(PK), name, shift, overtime_rate}
DRUG {drug_id(PK), name, description}
TREATMENT {treatment_id(PK), treatment_name, description}

PATIENT (N) ==ADMITTED_IN== (1) WARD  [total from PATIENT]

DOCTOR (M) --ASSIGNED_TO-- (N) WARD  [M:N]

HEALTHCARE_ASSISTANT (M) --ASSOCIATED_WITH-- (N) WARD  [M:N]

PATIENT (M) --TREATED_BY-- (N) DOCTOR  [M:N]

PATIENT (M) --PRESCRIBED-- (N) DRUG
  [attributes: times_per_day, duration_days, start_date]

PATIENT (M) --REQUIRES-- (N) TREATMENT
  [attributes: date, result, administered_by (doctor/HA)]

HEALTHCARE_ASSISTANT {salary, overtime_hours, overtime_rate}
  [Payment includes overtime: total_pay = base + overtime_hours * rate]
```

Q: An information system is to be designed for keeping the records of Universe Cup Cricket Tournament. There are 10 teams, each country sends 15 players and 4 other members. For players: runs and wickets. For non-players: role and years of experience. 38 matches, 11 referees. Draw E-R model.

Asked: 2072 Magh (Back Exam) | Marks: [8]

Answer

Entities and Attributes:

- TEAM: {team_id(PK), country_name, coach_name, manager_name}
- PLAYER: {player_id(PK), name, role (batsman/bowler/all-rounder), total_runs, total_wickets} — 15 per team
- NON_PLAYER_MEMBER: {member_id(PK), name, role(coach/manager/physio), years_of_experience} — 4 per team
- MATCH: {match_id(PK), match_date, venue, phase (group/knockout), result}

- GROUND: {ground_id(PK), name, city, seating_capacity, ground_size}
- REFEREE: {referee_id(PK), name, nationality, experience_years} — 3 referees per match
- PERFORMANCE: {perf_id(PK), runs_scored, wickets_taken} — player performance per match

Relationships:

- REPRESENTS: PLAYER (N) → TEAM (1); NON_PLAYER_MEMBER (N) → TEAM (1) — each belongs to one team [N:1, total]
- PLAYS: TEAM (M) ↔ MATCH (N) — two teams per match [M:N]; roles: team1, team2; attribute: score, result
- HELD_AT: MATCH (N) → GROUND (1) — each match at one ground [N:1, total from MATCH]
- OFFICIATED_BY: MATCH (M) ↔ REFEREE (N) — 3 referees per match [M:N]
- PERFORMANCE_IN: PLAYER records PERFORMANCE per MATCH [ternary or associative entity]

ER DIAGRAM — Textual Representation

```
TEAM {team_id(PK), country_name}
PLAYER {player_id(PK), name, role, total_runs, total_wickets}
NON_PLAYER {member_id(PK), name, role, years_experience}
MATCH {match_id(PK), match_date, phase, result}
GROUND {ground_id(PK), name, city, seating_capacity}
REFEREE {referee_id(PK), name, nationality}
PERFORMANCE {perf_id(PK), runs_scored, wickets_taken, catches}

PLAYER (N) ==REPRESENTS== (1) TEAM  [15 players per team]
NON_PLAYER (N) ==MEMBER_OF== (1) TEAM  [4 per team]

TEAM (M) ==PLAYS_IN== (N) MATCH
  [exactly 2 teams per match; roles: team1, team2; attribute: score]

MATCH (N) ==HELD_AT== (1) GROUND

MATCH (M) ==OFFICIATED_BY== (N) REFEREE  [3 referees per match]

PLAYER + MATCH ===RECORDS=== PERFORMANCE
  [Associative entity; PK: (player_id, match_id)]
```

Q: Draw a complete ER-diagram for: A Bus Company owns buses. Each bus allocated to a route. Each route passes through towns. Drivers allocated to each stage of a route. Some towns have a garage. Buses identified by registration number.

Asked: 2071 Bhadra (Regular/Back Exam) | Marks: [8+4]

Answer

Entities and Attributes:

- BUS: {reg_no(PK), capacity (single/double-deck), model, year}
- ROUTE: {route_id(PK), route_no, avg_passengers_per_day}

- TOWN: {town_id(PK), town_name, location}
- DRIVER: {driver_id(PK), employee_no, name, address, phone(optional)}
- GARAGE: {garage_id(PK), name, capacity} — weak entity of TOWN (only some towns have garages)
- STAGE: {stage_id(PK), stage_no, distance} — a segment of a route between two towns

Relationships:

- ALLOCATED_TO: BUS (N) → ROUTE (1) — each bus allocated to one route [N:1, total from BUS]
- PASSES_THROUGH: ROUTE (M) ↔ TOWN (N) — route passes through multiple towns; town on multiple routes [M:N]; attribute: order_in_route
- COVERS_STAGE: DRIVER (M) ↔ STAGE (N) — one or more drivers allocated per stage; driver covers multiple stages [M:N]
- CONSISTS_OF: ROUTE (1) → STAGE (N) — a route has multiple stages [1:N]
- CONNECTS: STAGE connects TOWN(from) to TOWN(to) — binary with roles
- HAS_GARAGE: TOWN (1) → GARAGE (1) — some towns have a garage [1:1, partial from TOWN]
- KEPT_IN: BUS (N) → GARAGE (1) — buses kept in garages [N:1, partial from BUS]

ER DIAGRAM — Textual Representation

```
BUS {reg_no(PK), capacity, model}
ROUTE {route_id(PK), route_no, avg_passengers_per_day}
TOWN {town_id(PK), town_name}
DRIVER {driver_id(PK), employee_no, name, address, phone}
[GARAGE] {garage_no(partial PK), name, capacity}  -- weak entity of TOWN
STAGE {stage_id(PK), stage_no, distance}

BUS (N) ==ALLOCATED_TO== (1) ROUTE  [total from BUS]

ROUTE (M) --PASSES_THROUGH-- (N) TOWN  [attribute: order_in_route]

ROUTE (1) ==CONSISTS_OF== (N) STAGE  [identifying]

STAGE (N) --FROM-- (1) TOWN  [departure town of stage]
STAGE (N) --TO-- (1) TOWN  [arrival town of stage]

DRIVER (M) --COVERS-- (N) STAGE
  [one or more drivers per stage; attribute: schedule_time]

TOWN (1) --HAS-- (1) [GARAGE]  [partial from TOWN]
  [only some towns have a garage; GARAGE is weak entity of TOWN]

BUS (N) --KEPT_IN-- (1) GARAGE  [partial from BUS]
```

Degree and Cardinality (Theory Part)

(See C5 above for complete theory.)

- Degree: Number of participating entity sets. ALLOCATED_TO (binary=2), COVERS (binary=2).
- Cardinality: 1:1, 1:N, M:N — see specific relationships above.

Q: Draw a complete ER-diagram for the following case: A lecturer (having ID, room_no) is responsible for organising a number of course modules. Each module has unique code, name. A module is composed of lectures. A lecture has time, room, date and is delivered by a lecturer. Students can attend lectures and register for modules. Lecturer acts as tutor for students.

Asked: 2071 Magh (Back Exam) | Marks: [8+4]

Answer

Entities and Attributes:

- LECTURER: {lecturer_id(PK), name, room_no, office_hours}
- MODULE: {module_code(PK), module_name, description, credits}
- LECTURE: {lecture_id(PK), date, time, room} — part of a module, delivered by a lecturer
- STUDENT: {student_id(PK), name, enrollment_date}

Relationships:

- ORGANISES: LECTURER (1) → MODULE (N) — each module organised by one lecturer [1:N, total from MODULE]
- COMPOSED_OF: MODULE (1) → LECTURE (N) — module composed of lectures [1:N, total from LECTURE]
- DELIVERS: LECTURER (1) → LECTURE (N) — a lecture delivered by one lecturer [N:1, total from LECTURE]; a lecturer can deliver multiple lectures (possibly from different modules)
- ATTENDS: STUDENT (M) ↔ LECTURE (N) — student attends lectures [M:N]
- REGISTERED_FOR: STUDENT (M) ↔ MODULE (N) — student registers for modules [M:N]; attribute: registration_date
- TUTORED_BY: STUDENT (N) → LECTURER (1) — each student has one tutor [N:1, total from STUDENT]

ER DIAGRAM — Textual Representation

```
LECTURER {lecturer_id(PK), name, room_no}
MODULE {module_code(PK), module_name, description, credits}
LECTURE {lecture_id(PK), date, time, room}
STUDENT {student_id(PK), name, enrollment_date}

LECTURER (1) ==ORGANISES== (N) MODULE
  [Total from MODULE — every module has an organising lecturer]

MODULE (1) ==COMPOSED_OF== (N) LECTURE
  [Total from LECTURE — every lecture is part of some module]

LECTURE (N) ==DELIVERED_BY== (1) LECTURER
  [Total from LECTURE — every lecture has a lecturer]
  [Note: Same lecturer may both ORGANISE a module AND DELIVER its lectures]

STUDENT (M) --ATTENDS-- (N) LECTURE  [M:N, partial both]

STUDENT (M) ==REGISTERED_FOR== (N) MODULE
  [attribute: registration_date; student must be registered for modules]

STUDENT (N) ==TUTORED_BY== (1) LECTURER
  [Each student has exactly one tutor; one lecturer tutors many students]
  [Total from STUDENT — every student has a tutor]
```

Generalization and Specialization (Theory Part)

(See C5 above for complete explanation.)

- Specialization: Top-down process — LECTURER could be specialized into FULL_TIME_LECTURER and PART_TIME_LECTURER with different attributes.
- Generalization: Bottom-up process — LECTURER and STUDENT could be generalized into PERSON with common attributes (id, name, email).

Q: Assume that at Pine Valley Furniture each product is comprised of at least three components and components are used to make one or many products. Components are used to make other components. Raw materials are also considered to be components. Draw an ER-diagram for this case.

Asked: 2070 Magh (Back Exam) | Marks: [8+4]

Answer

Entities and Attributes:

- PRODUCT: {product_no(PK), description, cost}
- COMPONENT: {component_no(PK), description, unit_of_measure, type (raw_material/manufactured)}

Key Observations:

- A PRODUCT is comprised of at least 3 COMPONENTS → M:N relationship MADE_OF between PRODUCT and COMPONENT
- A COMPONENT can be part of many products and a product has many components → M:N
- Components are used to make other components → recursive (unary) M:N relationship on COMPONENT
- Raw materials are also components → specialization of COMPONENT

Relationships:

- MADE_OF: PRODUCT (M) ↔ COMPONENT (N) — each product made of ≥3 components; component used in many products [M:N]; attribute: quantity_required
- USED_IN: COMPONENT (M) ↔ COMPONENT (N) — recursive M:N; a component can be made from other components; attribute: quantity

ER DIAGRAM — Textual Representation

```
PRODUCT {product_no(PK), description, cost}
COMPONENT {component_no(PK), description, unit_of_measure, type}
  Specialization (partial, overlapping):
    RAW_MATERIAL {supplier, material_grade}
    MANUFACTURED_COMPONENT {manufacturing_process}

PRODUCT (M) ==MADE_OF== (N) COMPONENT
  [Each product made of at least 3 components (total from PRODUCT)]
  [attribute: quantity_required]

COMPONENT (M) --USED_TO_MAKE-- (N) COMPONENT  [RECURSIVE / UNARY M:N]
```

```
   [A component is used as sub-component in another component]
   [attribute: quantity_needed, assembly_order]
   [Roles: 'sub_component' and 'parent_component']

 Assumption: A raw material (leaf component) is not made of other components.
   This is a tree-like recursive structure (Bill of Materials / BOM).
```

Total Participation in this Context:

- PRODUCT has total participation in MADE_OF — every product must have at least one component (in fact, at least 3 per the problem statement)
- COMPONENT has partial participation in MADE_OF as a parent — raw materials at the bottom of the BOM are not made of any other component

SUMMARY TABLE — Chapter 2 Question Coverage

The following table summarizes all Chapter 2 questions found in TU-IOE BCT examination papers from 2070–2082, their weightage, and coverage in this answer book.

No.	Question/Topic	Asked In	Marks
C1	Types of Data Models (logical, physical, conceptual, hierarchical, network, relational)	2073 Magh	[1+3]
C2	E-R Model concept, entities & entity sets, attributes & keys	Multiple years	Conceptual
C3	Types of Attributes & Discriminator Key	2080 Ashwin	[2+2]
C4	Strong and Weak Entity Sets; Total/Partial Participation	2078 Baishakh, 2078 Poush	[2+2]
C5	Relationship and Relationship sets; Degree; Cardinality	2077 Chaitra	[2+2]
C6	Generalization, Specialization, Aggregation in ER	2077 Chaitra, 2076 Baisakh	[2+2]
C7	Unary relationship; ER to Relational Schema	2075 Bhadra	[2+4]
C8	Discriminator in ER; Keys in database design	2074 Bhadra	[4]
C9	Network vs Relational Data Model	2072 Magh	[4]
ER1	Hospital Management System	2082 Kartik	[8]
ER2	Medium-Sized Company (interdisciplinary projects)	2081 Chaitra, 2079 Chaitra	[8]
ER3	Home Decor Company (furniture, salespersons, suppliers)	2081 Ashwin	[8]
ER4	ISA Hierarchy (employees, alumni, students)	2080 Ashwin	[8]
ER5	Procurement Department (ABC Organization)	2079 Ashwin	[8]

ER6	Online Auction System	2079 Jestha	[3+8]
ER7	University Registrar's Office	2078 Chaitra	[8]
ER8	Engineering Company (skills & machine-types, weak entity)	2078 Poush, 2070 Bhadra	[8+4]
ER9	Publishing Company (books, authors, editors, suppliers)	2078 Baishakh	[8]
ER10	Online Bookstore	2077 Chaitra	[8]
ER11	Pharmacy Chain	2076 Baisakh	[8]
ER12	University (departments, instructors, courses, students)	2075 Bhadra	[6]
ER13	NFL Database (teams, players, matches)	2075 Baisakh	[8]
ER14	Production Company (machining, fabrication, assembly)	2074 Bhadra	[8]
ER15	Airport Database (airplanes, models, technicians, controllers)	2073 Bhadra	[8]
ER16	Airlines System (customers, reservations, flights)	2073 Magh	[8]
ER17	Hospital/Ward System (patients, doctors, drugs, treatments)	2072 Ashwin	[8]
ER18	Universe Cup Cricket Tournament	2072 Magh	[8]
ER19	Bus Company (buses, routes, drivers, garages)	2071 Bhadra	[8]
ER20	University Lecture/Module System	2071 Magh	[8]
ER21	Pine Valley Furniture (products, components)	2070 Magh	[8]

Note: ER diagrams are presented in textual notation due to the limitations of word processor format. In the actual examination, draw proper ER diagram shapes: rectangles for entities, ellipses for attributes, diamonds for relationships, double rectangles for weak entities, double diamonds for identifying relationships, and ISA triangles for specialization/generalization.

— End of Chapter 2 Answer Bank —